

Agentic AI: Building Autonomous Systems for Real-World Applications

Module 2: Agent Architecture: Memory, Tools, and Reasoning Loops

Quick Reference Guide

Learning Outcomes

- Diagnose limitations of standard prompting across knowledge, reasoning, and computation tasks.
- Compare chain-of-thought and zero-shot prompting based on reasoning quality and accuracy.
- Decide which memory type an agent requires based on the task context.
- Explain how the ReAct framework integrates reasoning steps and actions in agent prompts.
- Refine the initial agent concept by incorporating reasoning loops, memory requirements, and tool usage.

Limitations of Prompting and Model Reasoning

- Standard prompting relies on single-turn or single-step instructions and often fails on tasks that require multistep reasoning verification or correction.
- Language models can produce fluent explanations that appear correct while still arriving at incorrect conclusions.
- Small changes in prompt wording can lead to significantly different outputs, revealing fragile and inconsistent reasoning behavior.
- Reasoning improvements from prompting are uneven across tasks and models and do not guarantee reliability.
- Language models optimize for the likelihood of text rather than correctness or task success.

Knowledge, Reasoning, and Computation Failures

- Model failures can be categorized as missing knowledge, flawed reasoning, or a lack of computational grounding.
- Knowledge limitations occur when required information is missing, outdated, or inaccessible.
- Reasoning limitations occur when the model has the necessary information but fails to draw correct conclusions from it.
- Computation limitations occur when tasks require exact calculation comparison or numerical grounding that language models do not reliably provide.

- Diagnosing the type of failure is necessary to determine the appropriate agent capability.

Zero-Shot and Chain-of-Thought Reasoning

- Zero-shot reasoning shows that language models can generate intermediate reasoning steps without examples.
- Chain-of-thought prompting improves performance by decomposing complex problems into smaller steps.
- Intermediate reasoning reduces cognitive load per step but does not ensure correctness.
- Generated reasoning is not evaluated by default, and errors can persist unnoticed.
- Reasoning generation and reasoning evaluation rely on different capabilities.

Verification and Feedback

- Verification evaluates candidate solutions and selects correct outcomes from imperfect reasoning traces.
- Correctness often emerges from comparison and selection rather than a single forward pass.
- External feedback enables detection and correction of errors that reasoning alone cannot catch.
- Feedback loops allow agents to revise decisions based on observed outcomes.

Tool Use and Computational Grounding

- Language models lack inherent numeric grounding and struggle with exact computation and comparison.
- Tool calling allows agents to delegate computation, retrieval, or external interaction to reliable systems.
- Tools operate outside the language model and return verified results.
- Agents decide when to use tools based on task requirements rather than calling them by default.
- Tool use extends agent capabilities beyond the model's training data.

Reasoning and Acting Loops

- Agent architectures operate as iterative loops rather than single-prompt responses.
- Reasoning guides action selection, while actions produce new observations.
- Observations update the agent's context and inform subsequent reasoning steps.
- This loop enables agents to gather information, adapt behavior, and correct mistakes over time.

ReAct Framework

- The ReAct framework integrates reasoning steps and actions within a single agent loop.
- Reasoning is treated as an internal action that guides external behavior.
- Actions interact with the environment through tools, APIs, or other systems.
- ReAct outperforms reasoning-only and acting-only approaches by combining both.

Memory in Agent Architectures

- Short-term memory tracks reasoning steps, tool outputs, and actions within a single task or session.
- Short-term memory is temporary and does not persist across tasks.
- Long-term memory stores information, reflections, or experiences across tasks.
- Memory enables agents to reuse information and avoid repeating past failures.

Agent Architecture as System Design

- Reliable agent behavior depends on system design rather than larger models or longer prompts.
- Reasoning loops, memory, and tools are added intentionally to address specific failure modes.
- Agent architectures address the limits of scaling by introducing structure feedback and grounding.